

Research paper

A lightweight malware classification method based on short bit sequence visualization

Junyu Zhang^{a,b}, Chun Guo^{a,b,*}, Guowei Shen^{a,b}, Yuan Ping^c, Yunhe Cui^{a,b}, Yi Chen^{a,b}^a Guizhou Provincial Laboratory of Big Data, College of Computer Science and Technology, Guizhou University, Guiyang 550025, PR China^b Engineering Research Center of Text Computing & Cognitive Intelligence, College of Computer Science and Technology, Guizhou University, Guiyang 550025, PR China^c School of Information Engineering, Xuchang University, Xuchang 461000, PR China

ARTICLE INFO

Keywords:

Malware classification
 Malware visualization
 Deep learning
 Convolutional neural network
 Lightweight model

ABSTRACT

With the rapid growth of malware, malware classification methods based on images and deep learning have attracted significant attention. However, achieving a good balance among storage overhead, computational complexity, efficiency, and accuracy remains a challenge for existing methods. To address this, we introduce the concepts of image generation efficiency and model classification efficiency to systematically analyze the factors influencing performance. We identify that reducing extra processing steps and input image dimensionality is critical. Based on these insights, we propose a malware visualization method that partitions the raw binary sequence into short bit-blocks and applies a Markov filling mode, generating compact, fixed-size images without disassembly. Furthermore, we propose a malware classification method based on short bit sequence visualization (MalCSBSV) that integrates this visualization technique with a lightweight Convolutional Neural Network (CNN) containing only 54 thousand parameters. Experimental results on the Microsoft Malware Classification Challenge (BIG 2015) dataset and the Blue Hexagon open dataset for malware analysis (BODMAS) demonstrate that MalCSBSV achieves accuracies of 99.02% and 98.34%, respectively. Compared with several existing methods, it significantly reduces the model size to 0.21 megabytes (MB) and the computational cost to 9.22 million floating-point operations (FLOPs), while maintaining a compact image size. By avoiding the overhead of disassembly, MalCSBSV offers a more favorable trade-off between efficiency and accuracy than instruction-based methods, making it a practical solution for large-scale malware classification.

1. Introduction

In recent years, malware attacks have emerged as a significant threat to cybersecurity. They have caused widespread and lasting impacts on enterprises, governments, and other institutions around the world. As attacks continue to evolve and change, need for malware defense has significantly increased. According to a 2025 report, malware attacks constitute a substantial 15% of the 135,225 recorded cyber incidents (Cyberdefense, 2024, 2025). Compared with the previous year, there was an increase of approximately 4000 malware attack incidents. This highlights the rapid growth in both the quantity and complexity of malware. Therefore, achieving accurate and efficient malware classification is crucial for defending against increasingly sophisticated malware attacks.

There are two primary tasks involved in malware classification. The first is binary classification, which distinguishes between benign and malicious software to prevent the propagation of malware. The

second is multi-class classification, which identifies whether a malicious sample belongs to a known malware family. This facilitates targeted analysis and enhances defense effectiveness against specific threats (Raff and Nicholas, 2020). Existing malware analysis techniques include static analysis (Ni et al., 2018; Rhode et al., 2018; Ajiri et al., 2020; Feng et al., 2024; Coscia et al., 2025) and dynamic analysis (Salehi et al., 2017; Leon et al., 2021; Trizna et al., 2024; Yan et al., 2025; Zhen et al., 2025). While effective, static analysis based on operation codes (opcodes) and API sequences is bottlenecked by slow disassembly. This computational burden degrades speed and hinders practical application in large-scale scenarios. Dynamic analysis, though capable of revealing runtime behavioral patterns, requires controlled execution environments and incurs significant time overhead. In recent years, the application of deep learning (DL) in malware analysis has steadily increased. Existing works have demonstrated the feasibility of

* Corresponding author.

E-mail address: gc_gzedu@163.com (C. Guo).<https://doi.org/10.1016/j.engappai.2026.115559>

Received 31 October 2025; Received in revised form 11 June 2026; Accepted 27 June 2026

Available online 13 July 2026

0952-1976/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

converting malware into images and using DL models for accurate malware classification. This approach is known as malware classification based on images and DL (Xiao et al., 2021; Deng et al., 2023; Yuan et al., 2021; Shaukat et al., 2023; Kumar and Kumar, 2024), which falls under the scope of static analysis. It enables automated feature extraction and achieves high classification accuracy, thus attracting widespread attention.

Following this trend, researchers have proposed various malware classification approaches based on images and DL. These approaches generally fall into two categories: some use complex models for high accuracy, while others use lightweight architectures for faster classification. While model architectures are extensively optimized, input images are often constructed by directly mapping domain-conventional features (e.g., opcode sequences) or raw bytes via fixed 8-bit grayscale mapping, leaving their generation process and resulting spatial dimensions as under-optimized variables. Because existing methods often involve complex preprocessing and produce large-scale images, they impose heavy storage and computational burdens. Therefore, balancing these overheads against speed and accuracy remains challenging, making it essential to analyze the factors that drive pipeline efficiency for achieving a favorable trade-off.

Malware classification methods based on images and DL typically involve two main stages: converting a malware binary into a visual representation, and training a DL model for classification. To decompose the efficiency of these stages, we introduce three concepts: overall efficiency, image generation efficiency, and model classification efficiency (defined in Section 3). We then structure our analysis around two questions: (1) How do the image generation process and input dimensions drive the respective efficiencies of the image generation and classification stages? (2) How can high overall efficiency be achieved while balancing storage footprint, computational complexity, and accuracy?

To address these questions, we systematically analyzed both stages. Our investigation reveals that the number of extra processing steps (NEPS, defined in Section 3) critically impacts image generation efficiency. To minimize NEPS and reduce spatial dimensionality simultaneously, we propose a compact image representation generated directly from raw binaries using short bit-blocks and a Markov filling mode. For the classification stage, we design the Malware Short Bit Sequence Light Classification Network (MalSBSLCNet). Tailored specifically for small input dimensions, this lightweight model reduces parameters by simplifying redundant structures, particularly in fully connected layers. Together, these components eliminate unnecessary preprocessing and structural overhead, achieving a favorable balance among storage, speed, and accuracy. Our main contributions are summarized as follows:

(1) We decompose the pipeline of malware classification based on images and DL into image generation and model classification stages to analyze its efficiency. Our analysis reveals that the choice of extracted content and the resulting NEPS critically determine image generation efficiency. We also show that input dimensions directly impacts classification efficiency.

(2) We propose a malware visualization method by partitioning the raw binary sequence into short bit-blocks and applying a Markov filling mode, resulting in the Short Bit Sequence Markov Image (SBSMI). Unlike variable-length grayscale images whose storage grows with file size, SBSMI produces a compact, fixed-dimensional representation, yielding a much smaller and predictable storage footprint. It also generates faster than techniques relying on assembly instructions or opcodes.

(3) We design MalSBSLCNet, a lightweight CNN tailored for compact SBSMIs. In contrast to conventional methods that use complex models for large input dimensions, MalSBSLCNet significantly reduces the number of parameters, thereby achieving high classification efficiency.

(4) We propose MalCSBSV, a malware classification method based on short bit sequence visualization that integrates SBSMI with the

MalSBSLCNet architecture. Evaluated on the Microsoft Malware Classification Challenge dataset (BIG 2015) (Panconesi and Marian, 2015) and Blue Hexagon open dataset for malware analysis (BODMAS) (Yang et al., 2021), MalCSBSV offers a favorable trade-off between resource efficiency and classification accuracy, making it highly applicable for large-scale malware classification.

The rest of this paper is organized as follows. Section 2 reviews related work. Section 3 presents the motivation behind our approach. The proposed MalCSBSV framework is detailed in Section 4. Section 5 presents the experimental results, followed by a discussion in Section 6. Finally, Section 7 concludes the paper.

2. Related work

2.1. Malware classification methods based on traditional static and dynamic features

Depending on whether malware execution is required, features are generally categorized into two types: dynamic and static. Traditional malware classification methods typically use these features within classical machine learning frameworks. The following sections review the existing literature by categorizing these approaches into dynamic feature-based and static feature-based methods.

2.1.1. Dynamic feature-based methods

Dynamic feature analysis focuses on observing runtime behavior. It involves executing malware in a controlled environment to extract behavioral features. The extracted features typically include function call data (e.g., API and system calls) (Sikorski and Honig, 2012; Barker, 2021; Trizna et al., 2024; Coscia et al., 2025), operating system-level information (e.g., process information, performance counters, and system changes) (Tang et al., 2014; Berlin et al., 2015; Barker, 2021; Zhen et al., 2025), and network traffic data (Nari and Ghorbani, 2013; Bekerman et al., 2015). San et al. (2018) extracted API call traces from Cuckoo sandbox reports across 11 malware families and constructed a malware family classification system by integrating Random Forest (RF), K-Nearest Neighbors (KNN), and decision tables. Amer and Zelinka (2020) investigated the contextual relationships between API calls to differentiate between benign and malicious behaviors. Jimenez and Goseva-Popstojanova (2019) extracted power consumption and network traffic features and utilized supervised machine learning algorithms for classification. Engel et al. (2020) utilized 79 network flow features extracted via neural network analysis to classify malware. Trizna et al. (2024) developed a Transformer-based self-attention architecture that fuses heterogeneous dynamic log data (API calls, file/network operations) to detect and classify Windows malware, achieving improved performance and reduced training data requirements. S. Zhang et al. (2025) integrated static function calls and dynamic API call sequences to perform Android malware detection.

These studies demonstrate that dynamic features can be effectively used to detect and classify malware. However, dynamic feature-based methods typically rely on sandbox environments to extract runtime behavioral features. Certain malware samples possess anti-sandbox capabilities and can detect the execution environment. Consequently, accurately capturing the behavioral characteristics of such malware through dynamic analysis within sandbox environments remains a significant challenge (Egele et al., 2008; Blackthorne et al., 2016).

2.1.2. Static feature-based methods

Static analysis involves extracting features from malware samples without execution. These features are typically classified into disassembled and non-disassembled types based on the necessity of disassembly. Non-disassembled features include raw byte data (e.g., byte sequences, strings) (Raff et al., 2018; Islam et al., 2010), file header information (Raff et al., 2017; Karl-Bridge-Microsoft, 2024; Carrera Ventura, 2023), and file entropy (Han et al., 2015; Xiao et al., 2020). Since these

features bypass the need for complex code analysis, they offer the advantages of rapid extraction and procedural simplicity. Conversely, disassembled features are derived through reverse engineering techniques, involving the analysis of assembly instructions (e.g., opcodes, instruction sequences) and structural or semantic information (e.g., API calls, FCG, SysCall sequences) (Ferrand and Filiol, 2016). As a result, these features typically demand higher computational resources and longer processing times. Sun et al. (2017) proposed a malware family classification method based on static feature extraction, utilizing bytecode, assembly code, and PE features for classification. Hassen et al. (2017) explored machine learning features extracted via static analysis to classify malware binaries. Djaneye-Boundjou et al. (2019) extracted assembly language opcodes from malware and used the latent Dirichlet allocation algorithm for document modeling, followed by a k-nearest neighbor classifier. El Neel et al. (2022) compared various feature extraction and classification techniques to identify the most effective approach for real-time malware detection. Their study utilized static PE header features and image-based Fast Fourier Transform features. Feng et al. (2024) proposes a malware detection framework that integrates API documentation semantics with API call graphs using a graph neural network to enhance detection performance. Coscia et al. (2025) introduces an API-based automatic rule generation framework for YARA, which enhances the automation capability and accuracy of malware detection.

Compared to non-disassembled feature-based methods, disassembled feature-based methods offer more fine-grained insights into malware, such as opcode sequences and API calls. However, they must be extracted by parsing executable files using disassembly tools, which renders the extraction process more time-consuming and resource-intensive.

2.2. Malware classification methods based on images and DL

To address the limitations of manual feature engineering and enhance classification accuracy, malware classification methods based on images and DL have received increasing attention in recent years. These methods typically convert malware samples into visual representations and use DL techniques to automatically extract features for binary or multi-class classification. Existing studies have demonstrated that these image-based approaches can achieve promising performance. Based on the data source used for visualization, these methods can be broadly classified into two categories: raw byte-based methods and disassembly-based methods.

Raw byte-based methods generate images directly from binary files without requiring reverse engineering. A common approach involves converting byte sequences into grayscale images. For instance, Rezende et al. (2017) and Kalash et al. (2018) used ResNet-50 and VGG16, respectively, to classify such grayscale images. Beyond simple conversion, Wang et al. (2019) proposed mapping malware binaries into 3-channel RGB images to enrich feature representation, while Xiao et al. (2021) incorporated PE section distribution information into grayscale images and utilized a VGG16-SVM hybrid for classification. Xiao et al. (2020) introduced malware entropy maps to visualize statistical complexity, though the specific impact of segmentation granularity on performance requires further study. Additionally, Yuan et al. (2020) and Yuan et al. (2021) explored converting bytecode into Markov images. To address efficiency concerns, recent studies have focused on lightweight frameworks. Mosleh and Sharifian (2024) presented HCDNN, a distributed DL framework for hierarchical deployment across IIoT devices, and D. Zhang et al. (2025) proposed IMCMK, which utilizes multi-scale convolution kernels and channel attention mechanisms. While these methods validate the utility of visualization techniques, they predominantly focus on refining feature representation or improving network architectures. Consequently, the trade-off between generated image size, computational cost, and classification accuracy has received less attention in the design of these methods.

In contrast, disassembly-based methods rely on instruction-level features (e.g., opcodes) extracted via disassembly tools. Although these methods offer richer semantic information, they incur higher extraction costs compared to raw byte-based approaches. Ni et al. (2018) introduced an algorithm combining SimHash and CNN by transforming disassembled code into grayscale images; however, the issue of information redundancy or loss due to image scaling was not fully addressed. Darem et al. (2021) presented a visualization method based on opcode features for variant detection. Additionally, Jeon et al. (2024) designed a system that extracts multiple static features (bytes, opcodes, API calls) for IoT malware detection. While these disassembly-based approaches provide greater characterization capabilities, their dependence on time-consuming disassembly processes poses a challenge to extraction efficiency.

Overall, current research faces a trade-off: unprocessed byte-based methods (directly using raw bytes) offer high efficiency but limited semantic context, while disassembly-based methods provide rich semantics at the cost of speed. Achieving a balance among storage footprint, computational complexity, speed, and accuracy remains a significant challenge. To address this, this paper focuses on optimizing both image generation and the classification model. For image generation, we utilize fixed-length short bit sequences with a Markov filling mode to ensure efficiency. Regarding the classification model, we develop a lightweight CNN specifically designed for small-sized images, aiming to achieve an optimal trade-off between performance and resource consumption.

3. Motivation

3.1. Overview of efficiency analysis

Existing malware classification methods based on images and DL often face challenges in simultaneously balancing storage footprint, computational complexity, efficiency, and accuracy, typically prioritizing specific metrics at the expense of others. This work aims to develop a method that achieves an effective trade-off among these dimensions. To guide this design, we analyze the factors influencing efficiency and introduce specific metrics for evaluation, including overall efficiency, image generation efficiency, model classification efficiency, and NEPS.

Definition 1 (Overall Efficiency). Overall efficiency refers to the total time required for a malware classification method based on images and DL to provide a family label for a binary executable sample through end-to-end computation. It comprises image generation efficiency and model classification efficiency.

Definition 2 (Image Generation Efficiency). Image generation efficiency refers to the time taken to convert a single executable sample into an image representation suitable for classification. This includes all preprocessing, content extraction, and conversion steps necessary to prepare the sample for input into a DL model.

Definition 3 (Model Classification Efficiency). Model classification efficiency refers to the time it takes for the model to predict the family label from a given image characterizing a malware sample.

Definition 4 (Number of Extra Processing Steps, NEPS). NEPS refers to the number of processing steps necessary to extract content from the original binary executable sample, excluding the initial file reading operation, which is considered a standard step and therefore not counted in the NEPS.

Based on these definitions, we next analyze the main factors affecting image generation efficiency and model classification efficiency, respectively.

Table 1

The extraction efficiency of different extracted contents.

Extracted content	NEPS	Efficiency grade
Byte sequence	NEPS = 0	Highest
Bit sequence		
Meta data of file	NEPS ≥ 1	High
Instruction sequence	NEPS ≥ 2	Low
Opcode sequence		

3.2. Analysis of image generation efficiency

To analyze image generation efficiency, we examine extraction efficiency and filling efficiency separately, as the total time is the sum of these two components: Image Generation Time = Extraction Time + Filling Time.

When analyzing extraction efficiency, we focus on the extracted content, as it directly determines the NEPS required. Extraction time is influenced by the processing method used to extract content from an executable program. To evaluate differences in extraction time across various content types, we analyze the NEPS values required for obtaining different types of extracted content. For instance, if a method needs to obtain section information from a PE file, it must parse the PE file format at least once ($\text{NEPS} \geq 1$). If a method requires obtaining assembly instruction data, it must first parse the PE file to locate the code segment and then disassemble the code segment to extract the instructions ($\text{NEPS} \geq 2$). Table 1 compares the extraction efficiency for five types of extracted content. As shown in the table, extracting byte sequences or bit sequences requires only direct reading of the original binary file, without additional processing steps ($\text{NEPS} = 0$), resulting in the highest extraction efficiency. In contrast, extracting file metadata requires at least one parsing operation ($\text{NEPS} \geq 1$), leading to relatively high extraction efficiency. Extracting instruction sequences or opcode sequences necessitates both parsing and disassembly ($\text{NEPS} \geq 2$), resulting in low extraction efficiency. While instruction sequences or opcode sequences contain more semantic information than byte sequences or bit sequences, they involve NEPS greater than 2, meaning at least two additional processing steps are required, especially the disassembly step, causing extra computational overhead. However, in pursuit of higher accuracy, some recent malware classification methods based on images and DL have increasingly adopted instruction sequences or opcode sequences and therefore tend to exhibit low extraction efficiency. A detailed experimental analysis of this issue is provided in Section 5.3.4. Based on this analysis, using bit or byte sequences as extracted content is a highly efficient choice, particularly compared to methods requiring disassembly.

When analyzing filling efficiency, we focus on the filling mode. The common filling modes used in existing malware classification methods based on images and DL are Grayscale filling mode, Markov filling mode, and SimHash filling mode. When using the same extracted content, the efficiency differences among various filling modes are as follows: the Grayscale filling mode has the highest efficiency, as it only reshapes a one-dimensional sequence into a two-dimensional format without any additional processing steps. The Markov filling mode is the second most efficient; it requires traversing the data sequence to count transition frequencies between elements. The SimHash filling mode is the least efficient, as it relies on computationally intensive hash algorithms to generate the filling values. As the generated image dimensions increase, the required hash length also increases. To achieve the longer hash length, the outputs of multiple hash algorithms often need to be concatenated, further increasing computational overhead.

Given that malware samples belonging to the same family exhibit significant similarities in their binary content (manifested as byte or bit sequences), binary sequences are widely adopted as extracted content in malware classification methods based on images and DL. Specifically,

when using bit sequences, the bit-length of the data unit significantly impacts both the resulting image dimensions and classification accuracy. To analyze this relationship, we define the filling content as follows.

Definition 5 (Filling Content). Filling content refers to the data unit that either directly constitutes a pixel or serves as input for pixel computation, depending on the filling mode. The filling content length corresponds exactly to the bit-length of the data unit.

While Grayscale filling mode offers the highest filling efficiency, the resulting variable image dimensions require resizing for CNN compatibility, risking distortion of the original structural information. In contrast, the Markov filling mode generates images of fixed dimensions, making them directly suitable for CNN inputs without the need for resizing. Furthermore, this mode captures the statistical transition distribution of bit sequences, preserving sequential regularities critical for distinguishing malware families. For the Markov filling mode, the length of the filling content, denoted as l , plays a critical role in balancing computational cost and representation quality. As l increases, the state space size grows exponentially ($n = 2^l$), causing the image dimension ($n \times n$) and associated storage and computational costs to increase significantly. Fig. 1 illustrates the relationship between filling content length and image dimensions. However, selecting a suitable l is crucial for ensuring effective feature representation. A high l value leads to a high-dimensional state space and severe data sparsity. In such sparse spaces, numerous transition combinations occur rarely, rendering statistical features susceptible to interference from non-critical variations (e.g., address offsets) and lacking robustness. Conversely, a very low l value avoids sparsity but results in coarse feature resolution due to excessive quantization, which limits inter-class separability. To identify a suitable l that balances these trade-offs, Section 5.3.1 compares different values of l ranging from 4 bits to 8 bits and analyzes their impact on image dimensions, Floating Point Operations (FLOPs), end-to-end runtime, and accuracy.

3.3. Analysis of model classification efficiency

Model classification efficiency depends on both theoretical complexity (measured by FLOPs and number of parameters) and practical deployment overhead (inference time and memory usage). Among these factors, input image dimensions play a primary role. Based on the standard convolutional formula $FLOPs = 2HWK^2C_{in}C_{out}$, computational complexity scales linearly with spatial dimensions ($H \times W$). While techniques like global average pooling can decouple the fully connected layer parameters from input dimensions, larger images still proportionally expand the intermediate feature maps in earlier layers. Therefore, smaller inputs shrink the memory footprint of these activations. This alleviates memory bandwidth bottlenecks, thereby reducing inference time.

CNN architectures tailored to specific input characteristics are generally more efficient. A network designed for small inputs usually requires lower computational overhead than one built for larger resolutions. However, many existing malware visualization methods rely on an 8-bit mapping scheme, which corresponds to a state space of $2^8 = 256$. When constructing visual representations like transition matrices, this expands the input dimensions to 256×256 . Because computational complexity scales with image dimensions, processing these high-dimensional inputs generates substantial FLOPs and increases inference time, leaving clear room for efficiency improvements. Furthermore, simply increasing model complexity does not guarantee better classification results. In the context of an 8-bit mapping scheme, the expanded state space of 256 discrete states introduces substantial redundant states and background noise. Complex models applied to such high-dimensional data tend to overfit to these random variations rather than learning core features. Therefore, aligning model capacity with the actual information density of the input is crucial for improving both efficiency and accuracy.

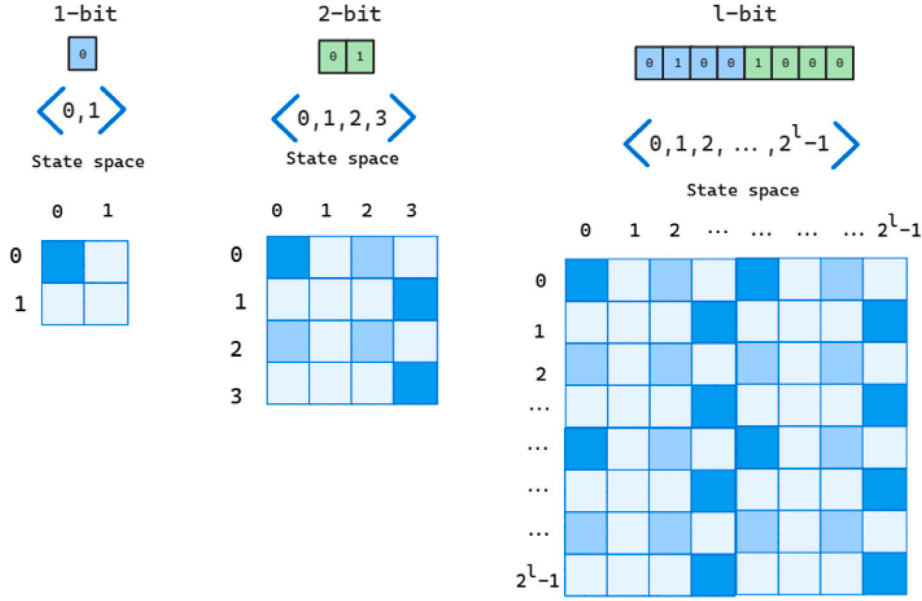


Fig. 1. Generated image dimensions as a function of filling content length l under Markov filling mode.

3.4. Solution idea

Based on the preceding analysis, we propose a design framework aimed at balancing storage footprint, computational complexity, efficiency, and accuracy. For the image generation stage, the design focuses on fast generation of compact, uniform images while retaining effective feature representation. To achieve this, our approach combines short bit blocks extracted directly from binary files with a Markov filling mode. Since these sequences are obtained without complex preprocessing, the NEPS is zero, resulting in high extraction efficiency. Additionally, the Markov filling mode captures the block-level transition statistics of the binary content, which are vital for distinguishing malware families. By selecting a suitable filling content length l , we generate compact, uniformly sized images that maintain high information density while avoiding the sparsity issues typical of 8-bit Markov image representations.

In the model classification stage, we design a lightweight CNN specifically tailored for these small-scale inputs. This compact architecture avoids the need for aggressive downsampling or large intermediate feature maps typical of models designed for larger images. By aligning the network's receptive field with the small input dimensions, we significantly reduce the FLOPs of the convolutional layers, achieving high classification accuracy with reduced deployment overhead. The specific details of the proposed method are presented in Section 4.

4. MalCSBSV

The overall framework of MalCSBSV is shown in Fig. 2. It mainly includes the following two stages.

Stage 1: Malware visualization based on short bit blocks. The input for this stage is an executable program. We first transform the file into a binary sequence of 0s and 1s. Using a predefined parameter l , this sequence is segmented into fixed-length l -bit subsequences. Next, we calculate the raw transition probabilities between adjacent subsequences to construct a Markov probability matrix. Since the matrix values are probabilities (ranging from 0 to 1), we map them to the range $[0, 255]$ to align with standard pixel intensity values. Finally, the resulting matrix is transformed into an SBSMI representing the program.

Stage 2: Malware image classification based on a lightweight CNN model. This stage uses a lightweight CNN model to perform multi-class classification on the SBSMIs generated in Stage 1. The model processes the input image and outputs a label identifying the malware family.

4.1. Malware visualization based on short bit blocks

In Stage 1, a fixed-dimension, compact SBSMI is generated without requiring disassembly. In malware research, variants within the same family typically share code and compilation environments. Even when processing raw binaries directly, the underlying Instruction Set Architecture (ISA) strongly influences the output, leading to consistent local bit patterns across variants. Absolute byte values are highly sensitive to data modifications (e.g., updating hardcoded C2 addresses). However, transition statistics between adjacent short bit-blocks capture the architecture and compilation-driven sequential regularities rather than exact values, making these statistics more resilient to these localized changes. Using short bit-blocks strikes an appropriate balance in granularity: they provide richer context than single-bit states, yet keep the state space small enough to maintain a compact transition matrix. Therefore, the shared codebase translates into similar transition probabilities, allowing the Markov image to act as a robust statistical descriptor for the family.

First, the executable program is read as a raw binary stream (i.e., the sequence of bits directly extracted from the file's binary data), resulting in a binary sequence of length N (where $N = 8 \times \text{file size in bytes}$), denoted as: $Bits = \{Bit_i \mid 0 \leq i < N\}$. This sequence is then divided into non-overlapping fixed-length subsequences of length l , starting from the first bit. If the final subsequence contains fewer than l bits (i.e., $N \bmod l \neq 0$), it is directly converted to an integer state, with the missing higher-order bits implicitly treated as 0. The resulting set of l -bit subsequences is denoted as: $lbits = \{lbit_i \mid 0 \leq i < \lceil N/l \rceil\}$.

Second, we construct a Markov matrix based on the transition probabilities between adjacent subsequences. The transition frequency $f(m, n)$ is counted by scanning the sequence of l -bit subsequences and recording how many times an l -bit subsequence $lbit_m$ is immediately followed by $lbit_n$. Since each element in the $lbits$ sequence has 2^l possible states, the dimensions of the Markov matrix are $2^l \times 2^l$. We calculate the transition probability $P_{m,n}$ from state m to state n using Eq. (1):

$$P_{m,n} = P(n|m) = \frac{f(m,n)}{\sum_{k=0}^{2^l-1} f(m,k)} \quad (1)$$

where $f(m, n)$ represents the raw transition frequency from $lbit_m$ to $lbit_n$. Note that if a state m does not appear in the sequence, the denominator becomes zero. In this case, we define the transition probability $P_{m,n}$ as

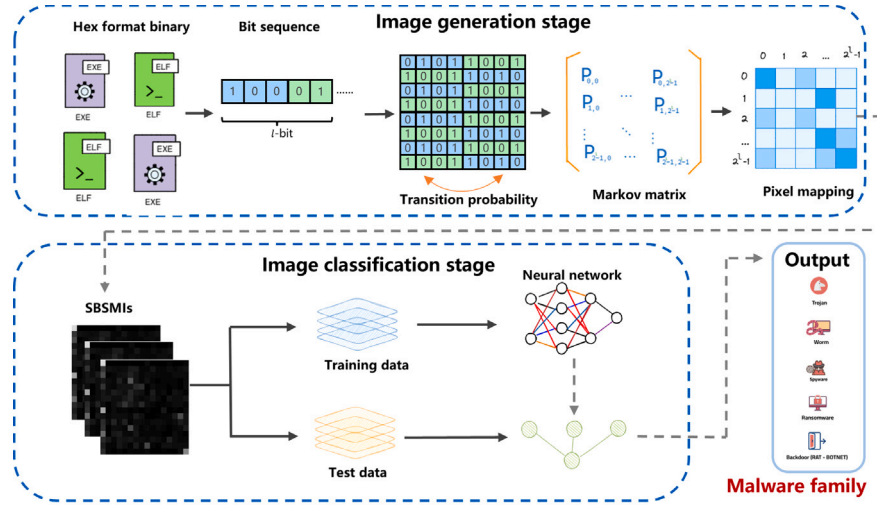
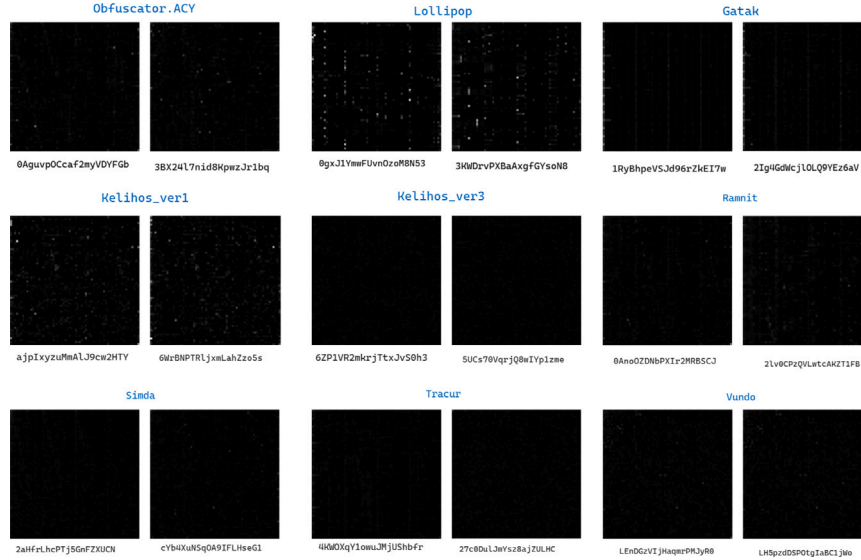


Fig. 2. Framework of MalCSBSV.

Fig. 3. SBSMIs generated from samples of different malware families ($l = 6$).

0, as transitions from an unobserved state are impossible. By calculating the probabilities for all state pairs, we obtain the final Markov matrix as shown in Eq. (2).

$$M = \begin{bmatrix} P_{0,0} & \cdots & P_{0,2^l-1} \\ P_{1,0} & \cdots & P_{1,2^l-1} \\ \vdots & \ddots & \vdots \\ P_{2^l-1,0} & \cdots & P_{2^l-1,2^l-1} \end{bmatrix} \quad (2)$$

Finally, since the transition probabilities P already range from 0 to 1, they are directly scaled to the standard grayscale pixel range $[0, 255]$ as follows: pixel value = $\lfloor P \times 255 \rfloor$. This converts the Markov matrix into an 8-bit grayscale image. Since the matrix dimension is determined by l , a smaller l value produces a more compact matrix and thus a smaller SBSMI. Algorithm 1 summarizes the SBSMI generation process.

To preliminarily validate the effectiveness of the proposed visualization method, we randomly selected two samples from each of the nine families in the BIG 2015 dataset and generated their SBSMIs with $l = 6$. As shown in Fig. 3, the generated 64×64 SBSMIs exhibit notable visual similarities within the same family, characterized by consistent spatial patterns. Conversely, SBSMIs from different families

display distinct visual differences. These clear intra-class consistencies and inter-class variations suggest that SBSMIs successfully capture family-specific transition statistics between adjacent short bit-blocks.

4.2. Malware image classification based on MalSBSLCNet

In Stage 2, we construct MalSBSLCNet, a lightweight CNN tailored for the compact SBSMIs generated in Stage 1. As depicted in Fig. 4, MalSBSLCNet consists of a feature extraction backbone and a classifier. The backbone first uses a 3×3 standard convolution with 16 channels, followed by Batch Normalization (BatchNorm), to capture initial features. Subsequently, six inverted residual modules extract deep features. The classifier utilizes Adaptive Average Pooling (AdaptiveAvgPool2d) to reduce spatial dimensions, followed by a flatten operation, a dropout layer with a rate of 0.4 to mitigate overfitting, and a final linear layer. By collapsing the spatial dimensions ($H \times W$) prior to classification, this design reduces the fully connected layer's parameter count, ensuring a compact model size.

MalSBSLCNet uses a dual-branch structure with 3×3 depthwise separable convolution (DWConv) and channel shuffle for efficient feature interaction. DWConv decomposes standard convolution into depthwise (channel-wise) and pointwise (1×1) operations, reducing FLOPs

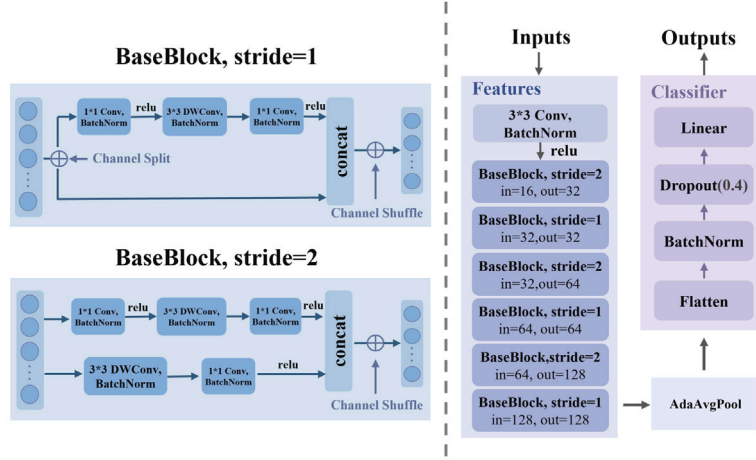


Fig. 4. Structure of the MalSBSLCNet.

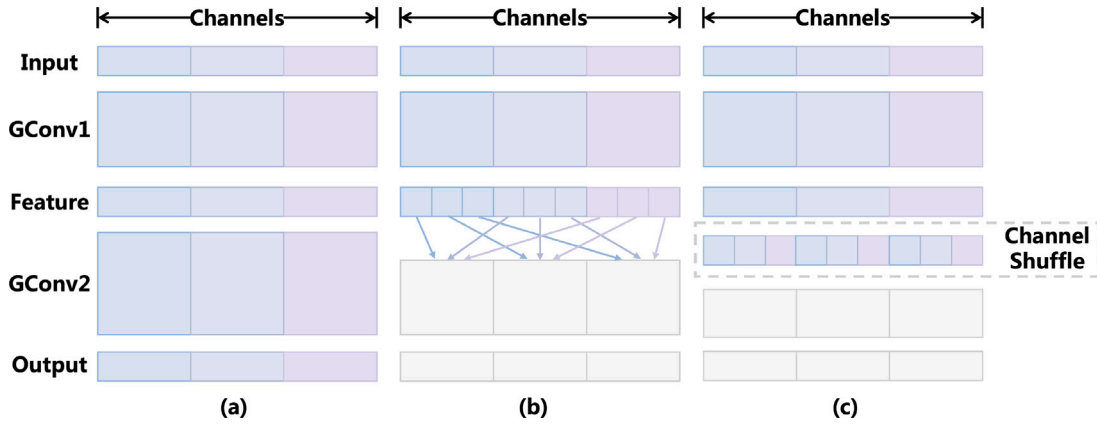


Fig. 5. The diagram of channel shuffle.

by nearly nine times. As illustrated in Fig. 5, the channel shuffle mechanism redistributes information across branches, resolving channel independence issues at zero extra cost. While derived from ShuffleNetV2, MalSBSLCNet is optimized for small inputs. With SBSMI inputs of 64×64 (where $l = 6$), the model requires only 9.22M FLOPs and 0.054M parameters. For baselines, we selected widely-used lightweight models like MobileNetV3-small (Table 2). Their computational costs (e.g., 61.46M FLOPs) are evaluated at their native 224×224 resolution. This native-to-native setup avoids interpolation noise from upsampling SBSMIs, and prevents architectural mismatches from forcing baseline models into 64×64 . Although differing native resolutions complicate direct FLOPs comparisons, evaluating each model at its intended scale reflects the actual computational cost in practical deployment. By removing the structural redundancy found in models built for larger inputs, MalSBSLCNet fits the compact input well. FLOPs are computed using the formulation in Eq. (3). The detailed architecture of MalSBSLCNet is provided in Appendix A.

The model is trained from scratch using the Adam optimizer and cross-entropy loss. Since weights are initialized randomly, we do not require pre-training or transfer learning.

5. Experimental evaluation

To evaluate the performance of MalCSBSV, we conduct experiments on the BIG 2015 and BODMAS datasets. We provide a detailed description covering four aspects: experimental setups, dataset details, evaluation metrics, and experimental results.

5.1. Experimental setup and dataset

Experimental environment: MalCSBSV is implemented in Python 3, utilizing PyTorch, PyTorch Lightning, and Scikit-learn. For both MalCSBSV and all baselines, malware image generation was performed on a PC with an Intel Core i9-13900K CPU and 64 GB RAM, while model training and all subsequent evaluations (including efficiency benchmarks) were conducted on a server equipped with an Intel Xeon Platinum 8370C (2.8 GHz) CPU and an NVIDIA RTX A6000 GPU.

Dataset: We adopt the Microsoft BIG 2015 dataset and the public BODMAS dataset for evaluation. The BIG 2015 dataset comprises 10,868 malware samples from nine families, with each sample accompanied by an ASM file and a BYTE file. For BODMAS, we use 18,498 samples from 10 families. Detailed statistics regarding family names and sample counts are presented in Fig. 6. During the creation of the BIG 2015 BYTE files, IDA expands PE sections based on the memory layout, introducing undefined bytes represented by '??'. We excluded all '??' entries during preprocessing in the BYTE files, then used for MalCSBSV and all compared methods. It is critical to note that these '??' entries are not inherent characteristics of normal PE files; in raw PE files, such regions are either compactly stored or filled with standard null bytes. They are strictly parsing artifacts introduced by IDA's specific memory expansion. We calculated the proportion of '??' entries per family in BIG 2015. The results show an average proportion of 37.39% with a high standard deviation of 31.3%. Rather than indicating a bias from our removal, this high variance suggests that retaining these artifacts would severely skew the Markov statistics, as models would inadvertently learn the parsing discrepancies of IDA

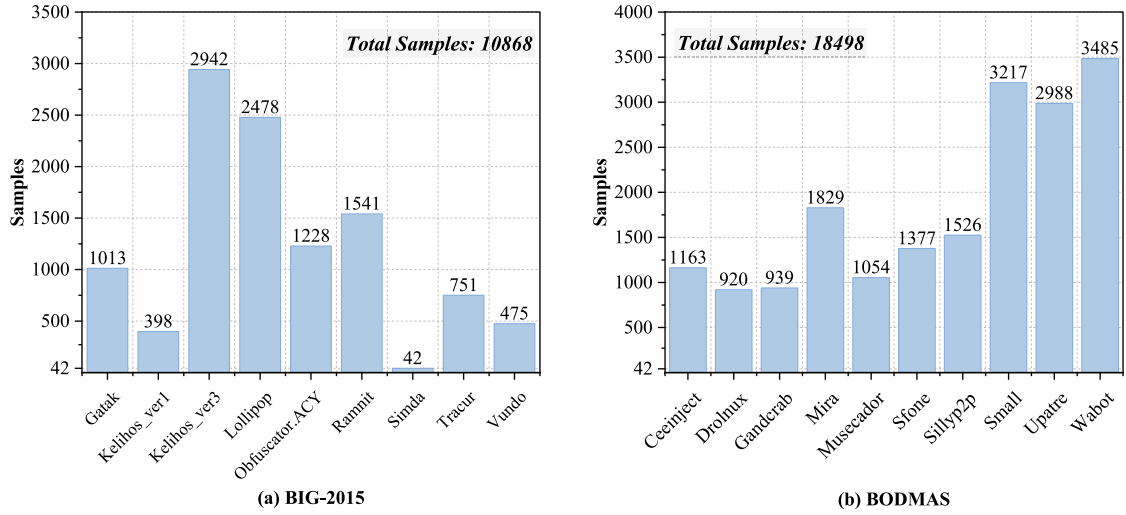


Fig. 6. Sample distribution in the experimental datasets: (a) BIG 2015 and (b) BODMAS.

Algorithm 1: Generating an SBSMI

```

Input : PE file,  $E$ ; filling content length,  $l$ 
Output: SBSMI,  $M$ 

/* Open a bit-level reader for file  $E$  */
1  $br \leftarrow \text{OpenBitReader}(E)$ ;
/* Create a transition matrix of size  $2^l \times 2^l$ ,
   initialized to 0 */
2  $m \leftarrow \text{NewMatrix}(2^l, 2^l)$ ;
/* Read the first  $l$  bits as the initial state */
3  $prev\_bits \leftarrow \text{ReadBits}(br, l)$ ;
4 if  $prev\_bits = \text{EOF}$  then
5   return Empty image;
6 end
7 while true do
8    $curr\_bits \leftarrow \text{ReadBits}(br, l)$ ;
9   if  $curr\_bits = \text{EOF}$  then
10    break;
11  end
/* Convert both  $l$ -bit states to integer indices */
12  $i \leftarrow \text{BitsConvDec}(prev\_bits)$ ;
13  $j \leftarrow \text{BitsConvDec}(curr\_bits)$ ;
14  $m_{i,j} \leftarrow m_{i,j} + 1$ ;
15  $prev\_bits \leftarrow curr\_bits$ ;
16 end

/* Row-wise normalization to obtain transition
   probabilities */
17  $m \leftarrow \text{LiNorm}(m, \text{row})$ ;
/* Scale probabilities to grayscale intensity range */
18  $m \leftarrow m \times 255$ ;
/* Convert matrix to SBSMI */
19  $M \leftarrow \text{MatrixConvImage}(m)$ ;
20 return  $M$ 

```

across different PE structures rather than meaningful bit sequences. This is further corroborated by the BODMAS dataset, where bytes are extracted directly from raw PE files without IDA expansion. BODMAS contains zero '??' entries, definitively confirming that the presence of '??' in BIG 2015 is a tool-specific artifact and that our exclusion is a necessary de-noising step.

Experimental setup: We used stratified 10-fold cross-validation. For the primary comparative analysis (Section 5.3.4), we repeated the experiments 10 times using different random seeds to account for stochasticity, reporting both accuracy and runtime as mean $\pm$

Table 2

Model complexity comparison.

Model	FLOPs (M)	Params (M)	Model storage footprint (MB)
MobileNetV2	326.22	2.235	8.53
MobileNetV3-small	61.46	1.527	5.83
MobileNetV3-large	233.57	4.214	16.07
ShuffleNetV2	151.69	1.259	4.80
SqueezeNetV1.1	335.13	1.234	4.71
GhostNet 1.0x	155.86	5.183	19.77
EfficientNet-B0	415.15	5.289	20.17
MalSBSLCNet	9.22	0.054	0.21

Note: Baselines are evaluated at their native input dimensions (224×224); MalSBSLCNet uses its native SBSMI dimensions (64×64 , $l = 6$).

Table 3

The hyperparameters of different methods.

Method	Optimizer	Learning rate	Weight decay	Batch size	Max epochs
GDMC	Adam	$1.0e-04$	$1.0e-06$	32	100
MCSC	Adam	$1.0e-04$	$1.0e-06$	32	100
MDMC	Adam	$1.0e-04$	$1.0e-06$	32	100
MCTVD	Adam	$3.0e-04$	$2.0e-07$	32	250
HCDNN	Adam	$1.0e-03$	–	32	225
IMCMK	DiffGrad	$2.0e-03$	–	32	200
MalCSBSV	Adam	$1.0e-04$	$1.0e-06$	32	100

standard deviation. For other analytical experiments (e.g., varying filling content lengths), single-run results are reported to emphasize relative trends. During runtime measurement, gradients were disabled (`torch.no_grad()`) for the classification stage. Specific hyperparameters for all methods, including optimizer, learning rate, weight decay, and training epochs, are detailed in Table 3. All models were trained with a batch size of 32 and a constant learning rate without scheduling.

5.2. Evaluation metrics

To comprehensively assess the storage footprint, computational complexity, overall efficiency, and classification capability of MalCSBSV, we adopt the following metrics: Image storage footprint, Model storage footprint, FLOPs, End-to-end runtime, and Accuracy.

Image storage footprint refers to the disk space occupied by a single generated image. Model storage footprint denotes the disk space required to store the trainable parameters of the CNN model. FLOPs measures the theoretical computational complexity, which is calculated

Table 4
Comparison of performance on MalCSBSV with different filling content lengths (l).

Dataset	Filling content lengths (l)	Dimensions	Average image storage footprint (KB)	FLOPs (M)	Average end-to-end runtime (ms)	Accuracy
BIG 2015	4	16×16	0.33	0.60	278.93	0.9779
	5	32×32	1.10	2.33	289.54	0.9876
	6	64×64	4.12	9.22	291.33	0.9908
	7	128×128	16.11	36.78	313.47	0.9892
	8	256×256	64.37	147.03	389.61	0.9899
BODMAS	4	16×16	0.33	0.61	979.21	0.9677
	5	32×32	1.10	2.33	1023.64	0.9784
	6	64×64	4.12	9.22	1250.21	0.9844
	7	128×128	16.18	36.78	1276.52	0.9827
	8	256×256	64.66	147.03	1435.93	0.9838

by accumulating the operations across all layers as shown in Eq. (3) (Hoffmann et al., 2022). End-to-end runtime represents the overall time required to process a single sample, encompassing both the image generation time and the model classification time. Accuracy quantifies the classification performance, defined as the ratio of correctly classified samples to the total number of samples, as formulated in Eq. (4) (Bishop, 2006). For each class, accuracy is the proportion of correctly classified samples (equivalent to recall). In addition, to comprehensively and fairly evaluate the performance across all malware families without being biased by class imbalance, we adopt macro-averaged Precision, Recall, and F1-score. The corresponding equations are presented in Eqs. (8)–(10).

$$\text{FLOPs} = \sum_{l \in L} 2H_l W_l C_{in,l} K_l^2 C_{out,l} \quad (3)$$

$$\text{Accuracy} = \frac{N_{\text{correct}}}{N_{\text{total}}} = \frac{\sum_{i=1}^N \mathbb{I}(y_i = \hat{y}_i)}{N} \quad (4)$$

$$\text{Precision}_i = \frac{TP_i}{TP_i + FP_i} \quad (5)$$

$$\text{Recall}_i = \frac{TP_i}{TP_i + FN_i} \quad (6)$$

$$F1_i = 2 \times \frac{P_i \times R_i}{P_i + R_i} \quad (7)$$

$$\text{Macro-Precision} = \frac{1}{N} \sum_{i=1}^N P_i \quad (8)$$

$$\text{Macro-Recall} = \frac{1}{N} \sum_{i=1}^N R_i \quad (9)$$

$$\text{Macro-F1} = \frac{1}{N} \sum_{i=1}^N F1_i \quad (10)$$

5.3. Experimental results

5.3.1. Evaluation of classification performance with varying filling content lengths

We evaluate the performance of MalCSBSV using SBSMIs generated with varying filling content lengths (l). The results on the BIG 2015 and BODMAS datasets are presented in Table 4.

As shown in Table 4, the filling content length l affects storage footprint, computational complexity, and classification performance. Specifically, as l increases from 4 to 8, the average image storage footprint rises from 0.33 KB to over 64 KB, FLOPs increase from approximately 0.60 M to 147 M, and end-to-end runtime increases from 278.93 ms to 389.61 ms on BIG 2015 (and from 979.21 ms to 1435.93 ms on BODMAS). Regarding classification accuracy, the best performance is achieved when l is set to 6, with accuracy reaching 0.9908 on BIG 2015 and 0.9844 on BODMAS. Lower values (e.g., $l = 4, 5$) result in lower accuracy due to limited feature resolution. Conversely, higher values (e.g., $l = 7, 8$) cause data sparsity in the state

space, as evidenced by the accuracy drop to 0.9892 on BIG 2015 at $l = 7$ despite increased computational cost.

To analyze this trade-off, we use the Pearson Correlation Coefficient (PCC, Eq. (11)) to quantify intra-family similarity and inter-family discrimination: SimScore (Eq. (13)) measures the average similarity among samples within the same family (higher values indicate better compactness), while DisScore (Eq. (14)) measures the average discrimination between samples from different families (higher values indicate better separation). As shown in Fig. 7, when l increases from 4 to 6, SimScore slightly decreases from 0.9710 to 0.8987 but remains relatively high, while DisScore rises to a peak of 0.6489; when further increased to 8, DisScore slightly drops to 0.6422. This trend reflects the trade-off between quantization precision and state space sparsity: low l compresses distinct transition patterns between adjacent short bit-blocks of different families into the same state (low Discrimination), while high l expands the state space rapidly, leading to sparse transition matrices (low Discrimination). At $l = 6$ (64-level quantization), the precision preserves inter-family differences (high Discrimination) while maintaining intra-family compactness (relatively high SimScore). Therefore, by constructing a 64×64 matrix that captures sufficient transition patterns between adjacent short bit-blocks without excessive sparsity, $l = 6$ provides a reasonable trade-off between storage footprint, computational cost, and classification accuracy, and we set it as the default for all subsequent experiments.

$$\text{PCC}(X, Y) = \frac{\sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})}{\sqrt{\sum_{i=1}^n (X_i - \bar{X})^2} \cdot \sqrt{\sum_{i=1}^n (Y_i - \bar{Y})^2}} \quad (11)$$

$$\text{Family_PCC_Matrix} = \begin{bmatrix} \text{PCC}_{1,1}^{\text{avg}} & \text{PCC}_{1,2}^{\text{avg}} & \dots & \text{PCC}_{1,m}^{\text{avg}} \\ \text{PCC}_{2,1}^{\text{avg}} & \text{PCC}_{2,2}^{\text{avg}} & \dots & \text{PCC}_{2,m}^{\text{avg}} \\ \vdots & & \ddots & \vdots \\ \text{PCC}_{m,1}^{\text{avg}} & \text{PCC}_{m,2}^{\text{avg}} & \dots & \text{PCC}_{m,m}^{\text{avg}} \end{bmatrix} \quad (12)$$

$$\text{SimScore} = 1 - \frac{1}{n} \sum_{i=1}^n \left(\frac{1 - M_{ii}}{2} \right)^2 \quad (13)$$

$$\text{DisScore} = 1 - \frac{1}{n(n-1)} \sum_{\substack{i=1 \\ i \neq j}}^n \left(\frac{1 + M_{ij}}{2} \right)^2 \quad (14)$$

5.3.2. Comparison of performance with and without Laplace smoothing

To validate the stability of our Markov transition representation, we conducted an ablation study on Laplace smoothing, particularly focusing on the optimal filling content length $l = 6$ identified in our prior analysis. This technique mitigates the issue of zero probabilities for rare transitions, which can lead to unstable or sensitive models, especially at higher filling content lengths.

Table 5 compares the performance of our method with and without Laplace smoothing on both BIG 2015 and BODMAS datasets. The results show that smoothing has a negligible impact on accuracy for $l = 6$: accuracy remains unchanged at 0.9908 on BIG 2015, and only slightly decreases from 0.9844 to 0.9843 on BODMAS (a 0.01% drop). This

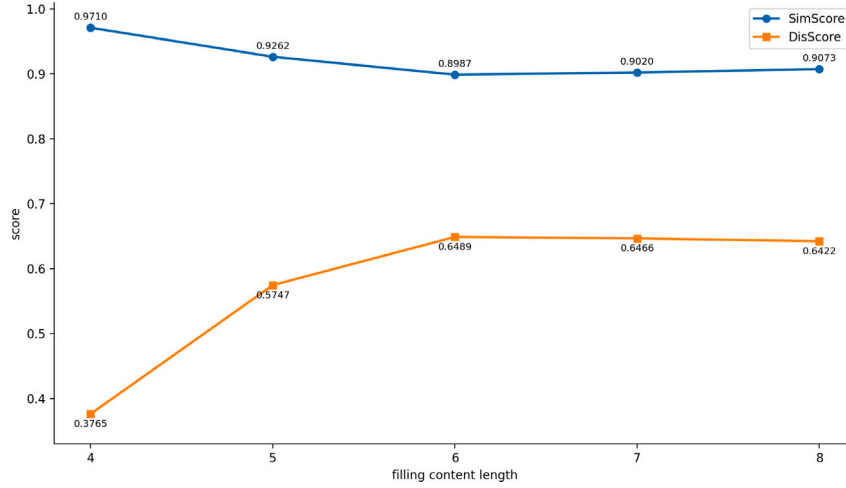


Fig. 7. Effect of filling content length l on SimScore and DisScore for BIG 2015 dataset.

Table 5

Performance comparison with and without Laplace smoothing across different filling content length settings on BIG 2015 and BODMAS datasets.

Dataset	l	Method	Acc	Macro precision	Macro recall	Macro F1
BIG 2015	6	Without smoothing	0.9908	0.9838	0.9918	0.9874
		With smoothing	0.9908	0.9844	0.9886	0.9862
	7	Without smoothing	0.9892	0.9891	0.9646	0.9697
		With smoothing	0.9890	0.9876	0.9618	0.9685
	8	Without smoothing	0.9899	0.9888	0.9641	0.9751
		With smoothing	0.9899	0.9910	0.9638	0.9761
BODMAS	6	Without smoothing	0.9844	0.9835	0.9847	0.9835
		With smoothing	0.9843	0.9836	0.9847	0.9837
	7	Without smoothing	0.9827	0.9829	0.9835	0.9830
		With smoothing	0.9827	0.9832	0.9833	0.9832
	8	Without smoothing	0.9838	0.9831	0.9838	0.9833
		With smoothing	0.9841	0.9834	0.9836	0.9835

confirms that the 64×64 state space is sufficiently dense to avoid zero-probability transitions, making Laplace smoothing unnecessary at this setting. However, when extending to higher filling content lengths ($l = 7, 8$), smoothing becomes beneficial: it maintains stable performance even at $l = 8$ (Table 5), where state space sparsity increases. This suggests that while our default method (without smoothing) is optimal for $l = 6$, applying Laplace smoothing would enhance robustness for larger l , though at the cost of reduced overall efficiency.

5.3.3. Comparative analysis of image representations using a fixed CNN backbone

To examine the impact of our image representation approach (SB-SMI) on performance, we conducted a controlled comparison using the same MalSBSLNet architecture for all image representations. Table 6 presents the performance of three representations (grayscale images, opcode-based SimHash, and our SBSMI) on both the BIG 2015 and BODMAS datasets. With the fixed MalSBSLNet model (64×64 input dimensions), SBSMI achieves the highest accuracy on both datasets. On BIG 2015, it reaches 0.9908, surpassing grayscale images (0.9455) and opcode-based SimHash (0.9826). On BODMAS, it achieves 0.9844, outperforming grayscale images (0.9437) and opcode-based SimHash (0.9789). Since the CNN architecture was held constant, these consistent improvements indicate that the SBSMI representation provides a more suitable input for the CNN.

5.3.4. Comparison with different malware classification methods based on images and DL

A. Performance Comparison on the BIG 2015 dataset

Table 7 presents a comparison between MalCSBSV and several representative peer methods on the BIG 2015 dataset. All selected baselines fall into the same domain, i.e., malware classification based on images and DL, but use varying image generation techniques and model architectures: GDMC (Kalash et al., 2018) (Grayscale images + VGG16), MDMC (Yuan et al., 2020) (raw byte-based Markov images + VGG16-like model), MCSC (Ni et al., 2018) (SimHash images from opcode sequences + CNN), MCTVD (Deng et al., 2023) (three-channel images from assembly instruction sequences + CNN), HCDNN (Mosleh and Sharifian, 2024) (Grayscale images + modified SqueezeNet), and IMCMK (D. Zhang et al., 2025) (Grayscale images + lightweight CNN). We evaluated these methods in terms of image storage footprint, model storage footprint, FLOPs, end-to-end time, and accuracy.

As shown in Table 7, MalCSBSV achieves a balanced performance between classification accuracy and computational overhead. In terms of accuracy, MalCSBSV reaches $99.02 \pm 0.26\%$, ranking second among all methods. This outperforms byte-based approaches such as GDMC ($92.16 \pm 0.35\%$) and HCDNN ($97.13 \pm 0.27\%$), and the improvement is statistically significant according to the Wilcoxon signed-rank test with Holm-Bonferroni correction ($p < 0.05$). While MCTVD achieves the highest accuracy ($99.16 \pm 0.29\%$), its performance is not statistically distinguishable from MalCSBSV ($p > 0.05$), and it relies on disassembly to extract assembly instructions, which introduces additional overhead. In contrast, MalCSBSV extracts features directly from binary bits, eliminating the need for reverse engineering and reducing computational costs. For lightweight deployment, MalCSBSV has the smallest model size at only 0.21 MB, substantially smaller than VGG16-based methods like GDMC (552 MB) and MDMC (524 MB). The average image storage footprint of MalCSBSV is 4.12 KB, which is two orders of magnitude smaller than grayscale-based methods (e.g., 778.44 KB). While MCSC

Table 6

Classification performance comparison of different image representations on MalSBSLCNet.

Dataset	Image representation	Acc	Macro precision	Macro recall	Macro F1
BIG 2015	Grayscale image	0.9455	0.8322	0.8218	0.8264
	Opcode-based SimHash	0.9826	0.9816	0.9639	0.9712
	SBSMI	0.9908	0.9838	0.9918	0.9874
BODMAS	Grayscale image	0.9437	0.9394	0.9318	0.9339
	Opcode-based SimHash	0.9789	0.9750	0.9812	0.9776
	SBSMI	0.9844	0.9835	0.9847	0.9835

Table 7

Comparison with different malware classification methods based on images and DL on the BIG 2015 dataset.

Method	Extracted content	Filling mode	Avg. Img. storage footprint (KB)	Model storage footprint (MB)	FLOPs (M)	Avg. E2E runtime (ms)	Accuracy (%)	Statistical significance (p < 0.05 after correction)
GDMC (Kalash et al., 2018)	Bytes	Grayscale	778.44	552	20 088.74	77.19 ± 8.19	92.16 ± 0.35	✓
MCSC (Ni et al., 2018)	Opcodes	SimHash	0.33	<u>1.32</u>	0.70	4183.23 ± 7.16	98.34 ± 0.37	✓
MDMC (Yuan et al., 2020)	Bytes	Markov	64.40	524	20 088.71	308.79 ± 5.13	98.37 ± 0.26	✓
MCTVD (Deng et al., 2023)	Instructions	Markov	11.40	25.2	206.41	21 358.16 ± 6.17	99.16 ± 0.29	
HCDNN (Mosleh and Sharifian, 2024)	Bytes	Grayscale	778.44	58.18	335.13	86.34 ± 7.39	97.13 ± 0.27	✓
IMCMK (D. Zhang et al., 2025)	Bytes	Grayscale	778.44	4.62	4233.22	<u>84.13 ± 5.76</u>	97.43 ± 0.38	✓
MalCSBSV	Bits	Markov	<u>4.12</u>	0.21	<u>9.22</u>	294.16 ± 5.19	<u>99.02 ± 0.26</u>	

Note: The best values are in bold, and the second-best values are underlined.

generates even smaller images (0.33 KB), it achieves a lower accuracy (98.34%) compared to MalCSBSV (99.02%).

In terms of computational complexity, MalCSBSV needs just 9.22M FLOPs, second only to MCSC (0.70 M). This is far less than the over 20,000 M FLOPs required by GDMC and MDMC. As for end-to-end time, grayscale image-based methods (GDMC, HCDNN, IMCMK) have the fastest average runtime (77.19 ± 8.19 ms, 86.34 ± 7.39 ms, and 84.13 ± 5.76 ms, respectively) due to direct byte-to-pixel mapping, though their classification accuracy is lower. Methods using assembly instructions or opcodes, such as MCTVD and MCSC, incur high runtime overhead (21358.16 ± 6.17 ms and 4183.23 ± 7.16 ms, respectively), making them less suitable for real-time tasks. MalCSBSV's average end-to-end time is 294.16 ± 5.19 ms. While it is slower than grayscale image-based methods, it is much faster than the methods using assembly instructions or opcodes (MCSC and MCTVD), as well as the Markov-based method (MDMC). This efficiency stems from using short bit-blocks (which reduces the size of the transition matrix) and the Markov filling mode, avoiding complex instruction parsing. Overall, MalCSBSV balances the high accuracy of semantic-rich methods with the efficiency of bit-based approaches, making it suitable for practical deployment.

The BIG 2015 dataset exhibits class imbalance, with the Simda family containing only 42 samples (the smallest class). The confusion matrices in Fig. 8 show that MalCSBSV and other methods all face challenges with the Simda class. For instance, HCDNN achieves the highest accuracy of 0.82 for Simda, while methods like GDMC and MDMC only achieve the accuracy of 0.12 and 0.55 for this class, respectively. Table 8 presents the per-class performance of MalCSBSV, including accuracy, precision, recall, and F1-score. For the Simda family, MalCSBSV achieves an accuracy of 0.7475, precision of 1.0000, recall of 0.7475, and F1-score of 0.8555. Notably, Simda is most frequently misclassified as Ramnit, with a misclassification rate of 15%. We attribute this to the similarity in Markov transition patterns between the two families, which is exacerbated by Simda's extremely limited sample size, thereby reducing the statistical reliability of its transition features. Other classes exhibit low misclassification rates (≤5%), with the remaining eight families all achieving accuracy, precision, recall, and F1-scores of 0.9723 or higher.

Table 8

Per-class performance of MalCSBSV on the BIG 2015 dataset.

Class	Acc	Precision	Recall	F1
Gatak	0.9856	0.9802	0.9856	0.9829
Kelihos_ver1	1.0000	0.9747	1.0000	0.9872
Kelihos_ver3	0.9932	0.9916	0.9932	0.9924
Lollipop	0.9919	0.9873	0.9919	0.9896
Obfuscator.ACY	0.9804	1.0000	0.9804	0.9901
Ramnit	1.0000	0.9832	1.0000	0.9915
Simda	0.7475	1.0000	0.7475	0.8555
Tracur	1.0000	0.9723	1.0000	0.9860
Vundo	1.0000	0.9817	1.0000	0.9908

To address the challenge of class imbalance, particularly for minority classes like Simda, we used the Synthetic Minority Over-sampling Technique (SMOTE) to enhance model performance. Specifically, we treated the output of the feature extraction stage as the high-dimensional embedding representation of samples and considered this space as the feature space. We applied SMOTE exclusively to the Simda class, which had the fewest samples, to perform data augmentation within this feature space.

In our experimental setup, we expanded the Simda samples in the training set by a factor of 5 and set $k_{\text{neighbors}} = 3$ for SMOTE. As shown in Table 9, SMOTE substantially improved the recognition of the Simda class: recall increased from 0.7475 to 0.8571, and the F1-score reached 0.9000. However, precision slightly dropped from 1.0000 to 0.9474 due to a minor increase in false positives. For other classes (e.g., Kelihos_ver3, Lollipop, Ramnit), metrics remained stable with only slight fluctuations, indicating that the targeted SMOTE strategy effectively improved minority class detection with marginal impact on majority classes.

B. Performance Comparison on the BODMAS dataset

Table 10 presents the performance of MalCSBSV alongside the same baseline methods on the BODMAS dataset. To ensure a consistent evaluation, we assessed the methods using the same metrics: image storage footprint, model storage footprint, FLOPs, end-to-end time, and accuracy.

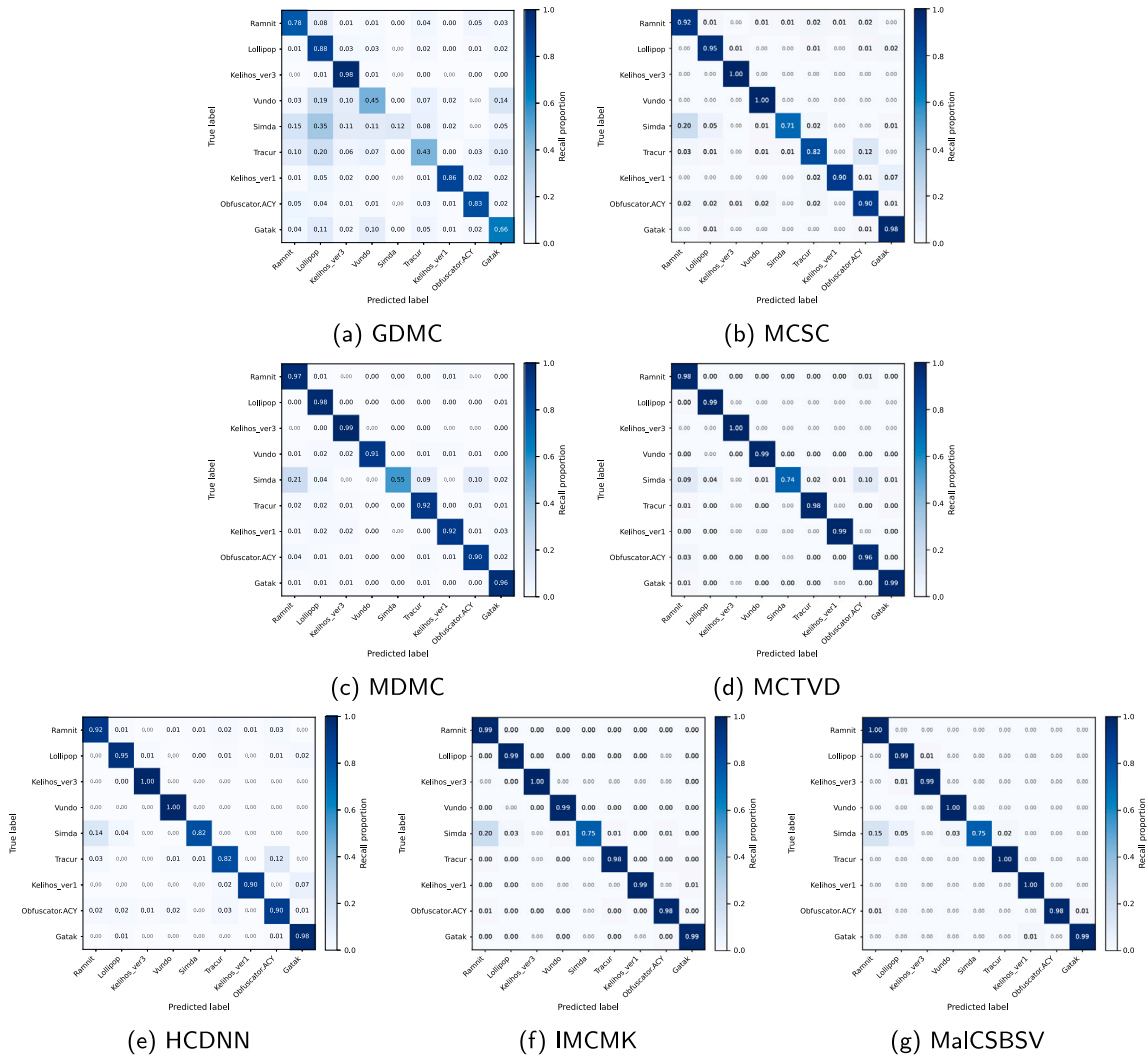


Fig. 8. Confusion matrices of different methods on the BIG 2015 dataset.

Table 9

Per-class performance of MalCSBSV on the BIG 2015 dataset with SMOTE.

Class	Acc	Precision	Recall	F1
Gatak	0.9862	0.9794	0.9862	0.9828
Kelihos_ver1	1.0000	0.9707	1.0000	0.9851
Kelihos_ver3	0.9942	0.9902	0.9942	0.9922
Lolipop	0.9927	0.9860	0.9927	0.9893
Obfuscator.ACY	0.9829	0.9975	0.9829	0.9902
Ramnit	1.0000	0.9803	1.0000	0.9900
Simda	0.8571	0.9474	0.8571	0.9000
Tracur	1.0000	0.9690	1.0000	0.9843
Vundo	1.0000	0.9794	1.0000	0.9896

As shown in Table 10, MalCSBSV reaches an accuracy of $98.34 \pm 0.36\%$ on the BODMAS dataset, ranking first among all methods. According to the Wilcoxon signed-rank test with Holm-Bonferroni correction, this accuracy is statistically significant ($p < 0.05$) when compared to most baseline methods, including GDMC, MCSC, MDMC, HCDNN, and IMCMK, though not distinguishable from MCTVD ($p > 0.05$). In terms of storage footprint, MalCSBSV still shows its advantage in model storage footprint and image storage footprint. It has the smallest model storage footprint (0.22 MB) and the second-smallest average image storage footprint (4.12 KB) among all compared methods, making it well-suited for environments with limited resources. For computational complexity, MalCSBSV needs only 9.22M FLOPs,

ranking second only to MCSC. This is much lower than the thousands of M FLOPs needed by GDMC and MDMC, showing that its network is efficient. For efficiency, MalCSBSV has a much lower end-to-end time (1246.52 ± 6.73 ms) compared to methods that rely on disassembly, such as MCSC (13589.32 ± 81.77 ms) and MCTVD (67359.47 ± 78.79 ms), and also lower than MDMC (1259.37 ± 6.13 ms). Although its end-to-end runtime is slower than the grayscale image-based methods (GDMC, HCDNN, IMCMK, around 325 ms), it is much faster than other Markov-based methods.

Fig. 9 presents the confusion matrices of different methods on the BODMAS dataset. MalCSBSV shows high recall rates across all ten malware families. For instance, as shown in Table 11, MalCSBSV achieves perfect recall (1.00) for DroLnuX, Gandcrab, Mira, Musecad, Sfone, and Wabot, while maintaining recall above 0.93 for the remaining families (Sillyp2p: 0.9415, Small: 0.9375). MalCSBSV maintains stable performance across all malware families, with no significant drops in recall.

5.3.5. Empirical analysis of robustness to packed malware

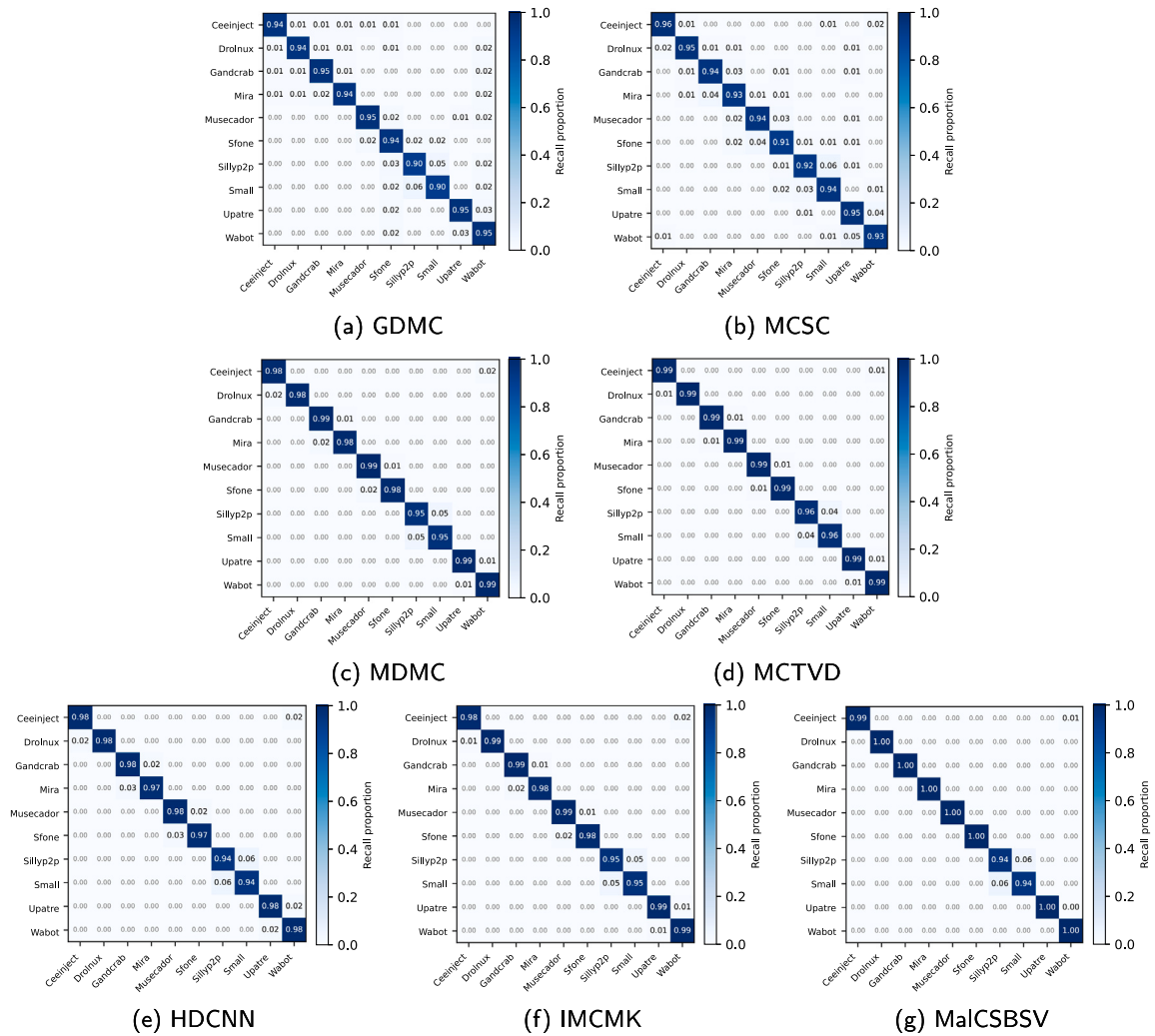
To evaluate how common packing techniques affect our method, we conducted an experiment on the BODMAS dataset using a 10-fold cross-validation strategy with fixed data splits (the BIG 2015 dataset was excluded as it lacks complete PE files required for packing operations). In each fold, the model was strictly trained on the original training samples. The test samples in each fold were evaluated under three

Table 10

Comparison with different malware classification methods based on images and DL on the BODMAS dataset.

Method	Extracted content	Filling mode	Avg. Img. storage footprint (KB)	Model storage footprint (MB)	FLOPs (M)	Avg. E2E runtime (ms)	Accuracy (%)	Statistical significance ($p < 0.05$ after correction)
GDMC (Kalash et al., 2018)	Bytes	Grayscale	2279.95	552	20 088.74	<u>327.14 ± 5.19</u>	94.17 ± 0.38	✓
MCSC (Ni et al., 2018)	Opcodes	SimHash	0.33	<u>1.32</u>	0.70	13 589.32 ± 81.77	97.34 ± 0.32	✓
MDMC (Yuan et al., 2020)	Bytes	Markov	64.40	524	20 088.71	1259.37 ± 6.13	<u>98.23 ± 0.37</u>	✓
MCTVD (Deng et al., 2023)	Instructions	Markov	11.40	25.2	206.41	67 359.47 ± 78.79	98.19 ± 0.36	
HCDNN (Mosleh and Sharifian, 2024)	Bytes	Grayscale	2279.95	58.18	335.13	327.55 ± 6.13	97.17 ± 0.31	✓
IMCMK (D. Zhang et al., 2025)	Bytes	Grayscale	2279.95	4.63	4233.22	319.93 ± 6.54	97.34 ± 0.34	✓
MalCSBSV	Bits	Markov	<u>4.12</u>	0.22	<u>9.22</u>	1246.52 ± 6.73	98.34 ± 0.36	

Note: The best values are in bold, and the second-best values are underlined.

**Fig. 9.** Confusion matrices of different methods on the BODMAS dataset.

distinct states: (1) original samples, (2) the same samples packed with UPX, and (3) the packed samples after standard UPX unpacking. Table 12 presents the averaged results across the 10 folds. As expected for static analysis methods, packing significantly degrades performance (accuracy drops from 0.9844 to 0.6751), as the packing process alters the statistical distribution of the block-level data (Liu et al., 2021).

However, unpacking restores the performance almost entirely (accuracy recovers to 0.9838). The minor fluctuation in detailed metrics between original and unpacked samples (e.g., Macro F1 shifting slightly from 0.9835 to 0.9842 due to minor bit-level remnants left by the unpacking process) is negligible. This suggests that the bit-block transition statistics captured by MalCSBSV represent the core characteristics of

Table 11

Per-class performance of MalCSBSV on the BODMAS dataset.

Class	Acc	Precision	Recall	F1
ceeinject	0.9868	1.0000	0.9868	0.9934
drolinux	1.0000	1.0000	1.0000	1.0000
gandcrab	1.0000	1.0000	1.0000	1.0000
mira	1.0000	1.0000	1.0000	1.0000
muscadador	1.0000	1.0000	1.0000	1.0000
sfone	1.0000	1.0000	1.0000	1.0000
sillyp2p	0.9415	0.8814	0.9415	0.9105
small	0.9375	0.9706	0.9375	0.9538
upatre	0.9967	1.0000	0.9967	0.9983
wabot	1.0000	0.9967	1.0000	0.9983

Table 12

Impact of packing and unpacking on MalCSBSV performance using the BODMAS dataset.

Setting	Acc	Macro precision	Macro recall	Macro F1
Original samples	0.9844	0.9835	0.9847	0.9835
Packed samples	0.6751	0.6063	0.6167	0.5988
Unpacked samples	0.9838	0.9818	0.9876	0.9842

Table 13Per-class performance of MalCSBSV on BODMAS dataset and unseen malware using $\tau = 0.80$.

Class	Acc	Precision	Recall	F1
ceeinject	0.9226	1.0000	0.9226	0.9597
drolinux	0.9783	1.0000	0.9783	0.9890
gandcrab	0.9894	1.0000	0.9894	0.9946
mira	0.9945	1.0000	0.9945	0.9973
muscadador	1.0000	1.0000	1.0000	1.0000
sfone	1.0000	1.0000	1.0000	1.0000
sillyp2p	0.7910	0.9378	0.7910	0.8582
small	0.9378	1.0000	0.9378	0.9679
upatre	0.9967	0.9490	0.9967	0.9722
wabot	0.9971	0.9943	0.9971	0.9957
unseen	0.5333	0.2333	0.5333	0.3246

the malware; once the packer is removed, these features remain intact and highly discriminative.

5.3.6. Scalability to open-set recognition

To investigate MalCSBSV's scalability to open-set recognition, we introduce a probability threshold mechanism to identify unseen malware families. During inference, if the maximum class probability of a sample falls below a predefined threshold τ , the sample is labeled as "unseen", indicating low similarity to any known training family.

We evaluate this mechanism using a 10-fold cross-validation setup on the BODMAS dataset. In each fold, the test set is constructed by combining the standard validation split (one fold) with 105 additional unseen malware samples. These unseen samples belong to three families (Trojan-Banker.Banker, Ransomware.Cerber, and Backdoor.Popwin, 35 samples each) collected from VX-Heaven and Virusshare, which are entirely excluded from the training set. Table 13 presents the per-class performance when τ is set to 0.80. The model maintains high classification accuracy for known families (e.g., muscadador and sfone achieve 1.0000 across all metrics), while the recall for "unseen" class stands at 0.5333. The threshold τ governs the trade-off between identifying unseen samples and preserving known family accuracy. As shown in Table 14, increasing τ to 0.85 shifts the model's bias toward predicting unseen classes. Consequently, the recall for the unseen class increases (from 0.5333 to 0.6857) as more true unseen samples are correctly identified. However, this reduces recall for certain seen families (e.g., sillyp2p drops from 0.7910 to 0.6212) and decreases precision for the unseen class (from 0.2333 to 0.2000), due to an increase in false positives where known samples are misclassified as unseen.

These results indicate that while the method can detect unseen samples to some extent, precisely categorizing zero-day families remains

Table 14Per-class performance of MalCSBSV on BODMAS dataset and unseen malware using $\tau = 0.85$.

Class	Acc	Precision	Recall	F1
ceeinject	0.9054	1.0000	0.9054	0.9504
drolinux	0.9783	1.0000	0.9783	0.9890
gandcrab	0.9894	1.0000	0.9894	0.9946
mira	0.9945	1.0000	0.9945	0.9973
muscadador	1.0000	1.0000	1.0000	1.0000
sfone	1.0000	1.0000	1.0000	1.0000
sillyp2p	0.6212	0.9595	0.6212	0.7542
small	0.9378	1.0000	0.9378	0.9679
upatre	0.9967	0.9644	0.9967	0.9803
wabot	0.9971	0.9943	0.9971	0.9957
unseen	0.6857	0.2000	0.6857	0.3097

inherently challenging, as reflected by the lower unseen-class metrics. Nonetheless, tuning τ provides a practical trade-off between preserving known family accuracy and flagging unseen threats, suggesting the method's potential utility in real-world scenarios.

5.3.7. Model deployment feasibility analysis

We evaluate inference overhead in terms of memory and latency to assess deployment feasibility. On the BIG 2015 and BODMAS datasets, MalCSBSV's peak GPU memory usage on the server described in Section 5.1 (Intel Xeon Platinum 8370C CPU, NVIDIA RTX A6000 GPU) is 85.70 MB and 146.01 MB, respectively.

To validate real-world performance in CPU-only environments without GPU acceleration, we further tested the model on a PC with an Intel i5-12450H CPU (16 GB RAM) using the BODMAS dataset in single-process mode with a batch size of 32. In this setup, MalCSBSV achieves an average single-sample inference time of 0.915 ms, satisfying real-time requirements for most practical applications. The peak memory footprint is 438.2 MB (dominated by Python and PyTorch system overhead, with the model itself accounting for a minor portion). During steady-state batch inference, the average additional memory overhead per sample is 0.74 MB, and the largest intermediate activation tensor occupies 8 MB. This low and stable marginal overhead suggests that MalCSBSV is highly efficient and readily deployable on commodity hardware.

6. Discussion

By combining a Markov filling mode for short bit blocks with a lightweight CNN architecture, MalCSBSV reduces computational overhead while maintaining high accuracy on the BIG 2015 and BODMAS datasets. Although our deployment feasibility analysis (Section 5.3.7) indicates low latency and memory usage, applying the method in the wild introduces operational challenges beyond raw resource consumption.

Because MalCSBSV relies entirely on static block-level statistics, it struggles against sophisticated evasion and open-set scenarios. As demonstrated in our packed malware experiments (Section 5.3.5), packing significantly degrades accuracy by altering the underlying bit distribution. While unpacking restores performance, relying solely on external unpacking introduces operational bottlenecks, as automated tools are computationally expensive and often fail against novel obfuscation techniques. To address this, one practical approach is to add a lightweight pre-filter before MalCSBSV. Using fast heuristics like entropy analysis, this filter could flag likely packed samples and divert only them for unpacking, letting plain binaries pass directly to MalCSBSV. This selective pipeline avoids the computational cost of universal unpacking without sacrificing performance on non-packed samples.

Beyond evasion, static methods inherently suffer from concept drift: malware distributions evolve over time, gradually diverging from the

fixed training data. In practice, new families constantly emerge, making open-set robustness essential to catch novel malware without constant retraining. Accurately identifying entirely novel families is a difficult challenge. Our open-set evaluation (Section 5.3.6) shows that tuning probability thresholds alone forces a difficult trade-off: raising the threshold to catch unseen samples floods the predictions with known misclassifications, while lowering it increases the risk of missing unseen samples. Static thresholds cannot resolve this on their own. One direction worth exploring is dynamic behavioral features—execution-level semantics could offer clearer separation between genuinely unseen families and known ones, rather than relying on statistical probabilities alone. For concept drift, continual learning is another option: update the model incrementally while keeping catastrophic forgetting in check.

7. Conclusion

In this paper, we investigated the trade-off between classification accuracy and computational efficiency in malware classification based on images and DL. Specifically, we analyzed the factors influencing efficiency across the two critical stages: image generation and model classification. Our analysis identified the NEPS as a critical determinant of generation efficiency and highlighted the impact of input image dimensions on computational complexity.

Guided by these insights, we proposed a malware visualization method using short bit blocks combined with a Markov filling mode to generate compact SBSMIs that reduce storage footprint and generation cost. To classify these images, we designed MalSBSLCNet, a lightweight CNN tailored for the SBSMIs. By simplifying redundant structures, MalSBSLCNet significantly reduces parameter counts and computational costs while maintaining representation capability. Experimental results on the BIG 2015 and BODMAS datasets demonstrate that the MalCSBSV framework achieves a favorable trade-off among storage footprint, computational complexity, and classification accuracy, suggesting the practicality of the short-bit-sequence strategy. However, reliance on static bit sequences introduces operational challenges in deployment, specifically the dependency on external unpacking for obfuscated samples and susceptibility to concept drift as malware distributions evolve. Future work will focus on integrating dynamic behavioral analysis to bypass unpacking bottlenecks, and adopting continual learning to incrementally adapt the model to novel families and shifting distributions.

CRedit authorship contribution statement

Junyu Zhang: Writing – original draft, Software, Methodology, Data curation. **Chun Guo:** Writing – review & editing, Methodology, Funding acquisition, Formal analysis, Conceptualization. **Guowei Shen:** Resources, Investigation, Funding acquisition, Formal analysis. **Yuan Ping:** Writing – review & editing, Methodology. **Yunhe Cui:** Writing – review & editing, Validation, Investigation. **Yi Chen:** Writing – review & editing, Validation, Investigation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The authors thank the anonymous referees for their valuable comments and suggestions, which improved the technical content and the presentation of the paper. This work is supported by the National Natural Science Foundation of China under Grant No. 62162009, Major Scientific and Technological Special Project of Guizhou Province under Grant No. Qian Ke He Zhongda Zhuanxiang Zi[2024]014, Big Data

and Network Security Innovation Team of of Guizhou Provincial High Education Institution under Grant No. [2023]052, the Postgraduate Education Reform and Quality Improvement Project of Henan Province under Grant No. YJS2025GZZ36, and the Graduate Education Reform Project of Henan Province under Grant No. 2025SJGLX278Y.

Appendix A. Supplementary data

Supplementary material related to this article can be found online at <https://doi.org/10.1016/j.engappai.2026.115559>.

Data availability

The BIG 2015 and BODMAS datasets are publicly available.

References

- Ajiri, V., Butakov, S., Zavorsky, P., 2020. Detection efficiency of static analyzers against obfuscated android malware. In: 2020 IEEE 6th Intl Conference on Big Data Security on Cloud (BigDataSecurity), IEEE Intl Conference on High Performance and Smart Computing (HPSC) and IEEE Intl Conference on Intelligent Data and Security. IDS, IEEE, pp. 231–234.
- Amer, E., Zelinka, I., 2020. A dynamic Windows malware detection and prediction method based on contextual understanding of API call sequence. *Comput. Secur.* 92, 101760.
- Barker, D., 2021. *Malware Analysis Techniques: Tricks for the Triage of Adversarial Software*. Packt Publishing Ltd.
- Bekerman, D., Shapira, B., Rokach, L., Bar, A., 2015. Unknown malware detection using network traffic classification. In: 2015 IEEE Conference on Communications and Network Security. CNS, IEEE, pp. 134–142.
- Berlin, K., Slater, D., Saxe, J., 2015. Malicious behavior detection using windows audit logs. In: Proceedings of the 8th ACM Workshop on Artificial Intelligence and Security. pp. 35–44.
- Bishop, C.M., 2006. *Pattern Recognition and Machine Learning*. Springer, New York, NY, USA.
- Blackthorne, J., Bulazel, A., Fasano, A., Biernat, P., Yener, B., 2016. {AVLeak}: fingerprinting antivirus emulators through {Black-Box} testing. In: 10th USENIX Workshop on Offensive Technologies. WOOT 16.
- Carrera Ventura, E., 2023. Pefile. URL: <https://github.com/erocarrera/pefile>. original-date: 2015-04-13T13:45:19Z.
- Coscia, A., Lorusso, R., Maci, A., Urbano, G., 2025. APIARY: An API-based automatic rule generator for yara to enhance malware detection. *Comput. Secur.* 153, 104397. <http://dx.doi.org/10.1016/j.cose.2025.104397>, URL: <https://linkinghub.elsevier.com/retrieve/pii/S0167404825000860>.
- Cyberdefense, O., 2024. Security navigator 2024: Latest cybersecurity threats and trends. URL: <https://www.orangeCyberdefense.com/global/security-navigator>.
- Cyberdefense, O., 2025. Security navigator 2025: Latest cybersecurity threats and trends. URL: <https://www.orangeCyberdefense.com/global/security-navigator>.
- Darem, A., Abawajy, J., Makkar, A., Alhashmi, A., Alanazi, S., 2021. Visualization and deep-learning-based malware variant detection using OpCode-level features. *Future Gener. Comput. Syst.* 125, 314–323.
- Deng, H., Guo, C., Shen, G., Cui, Y., Ping, Y., 2023. MCTVD: A malware classification method based on three-channel visualization and deep learning. *Comput. Secur.* 126, 103084.
- Djaneye-Boundjou, O., Messay-Kebede, T., Kapp, D., Greer, J., Ralescu, A., 2019. Static analysis through topic modeling and its application to malware programs classification. In: 2019 IEEE National Aerospace and Electronics Conference. NAECON, IEEE, pp. 226–231.
- Egele, M., Scholte, T., Kirda, E., Kruegel, C., 2008. A survey on automated dynamic malware-analysis techniques and tools. *ACM Comput. Surv.* 44 (2), 1–42.
- El Neel, L., Copiaco, A., Obaid, W., Mukhtar, H., 2022. Comparison of feature extraction and classification techniques of PE malware. In: 2022 5th International Conference on Signal Processing and Information Security. ICSPIS, IEEE, pp. 26–31.
- Engel, V.J.L., Joshua, E., Engel, M.M., 2020. Detection of cyber malware attack based on network traffic features using neural network.
- Feng, P., Gai, L., Yang, L., Wang, Q., Li, T., Xi, N., Ma, J., 2024. DawnGNN: Documentation augmented windows malware detection using graph neural. *Comput. Secur.* 140, 103788. <http://dx.doi.org/10.1016/j.cose.2024.103788>, URL: <https://www.sciencedirect.com/science/article/pii/S0167404824000890>.
- Ferrand, O., Filiol, E., 2016. Combinatorial detection of malware by IAT discrimination. *J. Comput. Virol. Hacking Tech.* 12 (3), 131–136.
- Han, K.S., Lim, J.H., Kang, B., Im, E.G., 2015. Malware analysis using visualized images and entropy graphs. *Int. J. Inf. Secur.* 14 (1), 1–14.

- Hassen, M., Carvalho, M.M., Chan, P.K., 2017. Malware classification using static analysis based features. In: 2017 IEEE Symposium Series on Computational Intelligence. SSCI, IEEE, pp. 1–7.
- Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., Casas, D.d.L., Hendricks, L.A., Welbl, J., Clark, A., et al., 2022. Training compute-optimal large language models. arXiv preprint arXiv:2203.15556.
- Islam, R., Tian, R., Batten, L., Versteeg, S., 2010. Classification of malware based on string and function feature selection. In: 2010 Second Cybercrime and Trustworthy Computing Workshop. IEEE, pp. 9–17.
- Jeon, J., Jeong, B., Baek, S., Jeong, Y., 2024. Static multi feature-based malware detection using multi SPP-net in smart IoT environments. IEEE Trans. Inf. Forensics Secur. 19, 2487–2500. <http://dx.doi.org/10.1109/TIFS.2024.3350379>.
- Jimenez, J.H., Goseva-Popstojanova, K., 2019. Malware detection using power consumption and network traffic data. In: 2019 2nd International Conference on Data Intelligence and Security. ICDIS, IEEE, pp. 53–59.
- Kalash, M., Rochan, M., Mohammed, N., Bruce, N.D.B., Wang, Y., Iqbal, F., 2018. Malware classification with deep convolutional neural networks. In: 2018 9th IFIP International Conference on New Technologies, Mobility and Security. NTMS, pp. 1–5. <http://dx.doi.org/10.1109/NTMS.2018.8328749>, URL: <https://ieeexplore.ieee.org/abstract/document/8328749>.
- Karl-Bridge-Microsoft, 2024. PE format. URL: <https://learn.microsoft.com/windows/win32/debug/pe-format>.
- Kumar, S., Kumar, A., 2024. Image-based malware detection based on convolution neural network with autoencoder in Industrial Internet of Things using Software Defined Networking Honeypot. Eng. Appl. Artif. Intell. 133, 108374. <http://dx.doi.org/10.1016/j.engappai.2024.108374>, URL: <https://linkinghub.elsevier.com/retrieve/pii/S0952197624005323>.
- Leon, R.S., Kiperberg, M., Leon Zabag, A.A., Zaidenberg, N.J., 2021. Hypervisor-assisted dynamic malware analysis. Cybersecurity 4 (1), 19.
- Liu, H., Guo, C., Cui, Y., Shen, G., Ping, Y., 2021. 2-SPIFF: a 2-stage packer identification method based on function call graph and file attributes. Appl. Intell. 51, 9038–9053. <http://dx.doi.org/10.1007/s10489-021-02347-w>.
- Mosleh, M.R.B., Sharifian, S., 2024. An efficient cloud-integrated distributed deep neural network framework for IoT malware classification. Future Gener. Comput. Syst. 157, 603–617.
- Nari, S., Ghorbani, A.A., 2013. Automated malware classification based on network behavior. In: 2013 International Conference on Computing, Networking and Communications. ICNC, IEEE, pp. 642–647.
- Ni, S., Qian, Q., Zhang, R., 2018. Malware identification using visualization images and deep learning. Comput. Secur. 77, 871–885.
- Panconesi, A., Marian, W.C., 2015. Microsoft malware classification challenge (BIG 2015). <https://kaggle.com/competitions/malware-classification>. (Accessed 27 September 2025).
- Raff, E., Nicholas, C., 2020. A survey of machine learning methods and challenges for windows malware classification. arXiv preprint arXiv:2006.09271.
- Raff, E., Sylvester, J., Nicholas, C., 2017. Learning the pe header, malware detection with minimal domain knowledge. In: Proceedings of the 10th ACM Workshop on Artificial Intelligence and Security. pp. 121–132.
- Raff, E., Zak, R., Cox, R., Sylvester, J., Yacci, P., Ward, R., Tracy, A., McLean, M., Nicholas, C., 2018. An investigation of byte n-gram features for malware classification. J. Comput. Virol. Hacking Tech. 14 (1), 1–20.
- Rezende, E., Ruppert, G., Carvalho, T., Ramos, F., De Geus, P., 2017. Malicious software classification using transfer learning of resnet-50 deep neural network. In: 2017 16th IEEE International Conference on Machine Learning and Applications. ICMLA, IEEE, pp. 1011–1014.
- Rhode, M., Burnap, P., Jones, K., 2018. Early-stage malware prediction using recurrent neural networks. Comput. Secur. 77, 578–594.
- Salehi, Z., Sami, A., Ghiasi, M., 2017. MAAR: Robust features to detect malicious activity based on API calls, their arguments and return values. Eng. Appl. Artif. Intell. 59, 93–102.
- San, C.C., Thwin, M.M.S., Htun, N.L., 2018. Malicious software family classification using machine learning multi-class classifiers. In: Computational Science and Technology: 5th ICCST 2018, Kota Kinabalu, Malaysia, 29–30 August 2018. Springer, pp. 423–433.
- Shaukat, K., Luo, S., Varadharajan, V., 2023. A novel deep learning-based approach for malware detection. Eng. Appl. Artif. Intell. 122, 106030. <http://dx.doi.org/10.1016/j.engappai.2023.106030>, URL: <https://linkinghub.elsevier.com/retrieve/pii/S0952197623002142>.
- Sikorski, M., Honig, A., 2012. Practical Malware Analysis: The Hands-On Guide to Dissecting Malicious Software. no starch press.
- Sun, B., Li, Q., Guo, Y., Wen, Q., Lin, X., Liu, W., 2017. Malware family classification method based on static feature extraction. In: 2017 3rd IEEE International Conference on Computer and Communications. ICC3, IEEE, pp. 507–513.
- Tang, A., Sethumadhavan, S., Stolfo, S.J., 2014. Unsupervised anomaly-based malware detection using hardware features. In: International Workshop on Recent Advances in Intrusion Detection. Springer, pp. 109–129.
- Trizna, D., Demetrio, L., Biggio, B., Roli, F., 2024. Nebula: Self-attention for dynamic malware analysis. IEEE Trans. Inf. Forensics Secur. 19, 6155–6167. <http://dx.doi.org/10.1109/TIFS.2024.3409083>, URL: <https://ieeexplore.ieee.org/abstract/document/10551436>.
- Wang, S.-W., Zhou, G., Lu, J.-C., Zhang, F.-j., 2019. A novel malware detection and classification method based on capsule network. In: International Conference on Artificial Intelligence and Security. Springer, pp. 573–584.
- Xiao, M., Guo, C., Shen, G., Cui, Y., Jiang, C., 2021. Image-based malware classification using section distribution information. Comput. Secur. 110, 102420.
- Xiao, G., Li, J., Chen, Y., Li, K., 2020. MalFCS: An effective malware classification framework with automated feature extraction based on deep convolutional neural networks. J. Parallel Distrib. Comput. 141, 49–58.
- Yan, P., Tan, S., Wang, M., Huang, J., 2025. Prompt engineering-assisted malware dynamic analysis using GPT-4. IEEE Trans. Dependable Secur. Comput. 1–17. <http://dx.doi.org/10.1109/TDSC.2025.3599004>, URL: <https://ieeexplore.ieee.org/abstract/document/11128946>.
- Yang, L., Ciptadi, A., Laziuk, I., Ahmadzadeh, A., Wang, G., 2021. BODMAS: An open dataset for learning based temporal analysis of PE malware. In: 2021 IEEE Security and Privacy Workshops. SPW, pp. 78–84. <http://dx.doi.org/10.1109/SPW53761.2021.00020>.
- Yuan, B., Wang, J., Liu, D., Guo, W., Wu, P., Bao, X., 2020. Byte-level malware classification based on markov images and deep learning. Comput. Secur. 92, 101740.
- Yuan, B., Wang, J., Wu, P., Qing, X., 2021. IoT malware classification based on lightweight convolutional neural networks. IEEE Internet Things J. 9 (5), 3770–3783.
- Zhang, D., Song, Y., Xiang, Q., Wang, Y., 2025. IMCMK-CNN: A lightweight convolutional neural network with Multi-scale Kernels for Image-based Malware Classification. Alex. Eng. J. 111, 203–220.
- Zhang, S., Su, H., Liu, H., Yang, W., 2025. MPDroid: A multimodal pre-training Android malware detection method with static and dynamic features. Comput. Secur. 150, 104262. <http://dx.doi.org/10.1016/J.COSE.2024.104262>.
- Zhen, Y., Tian, D., Fu, X., Hu, C., 2025. A novel malware detection method based on audit logs and graph neural network. Eng. Appl. Artif. Intell. 152, 110524. <http://dx.doi.org/10.1016/j.engappai.2025.110524>, URL: <https://linkinghub.elsevier.com/retrieve/pii/S095219762500524X>.